Months 1 to 3: PyTorch foundations

Quick reference · Senior Data Scientist roadmap · Tensors, autograd, training loops and the habits that keep PyTorch runs reproducible.

B23 PyTorch foundations

B23-L1 Tensors, shape, dtype and shared memory

Convert to float32 at the edge, know which operations share memory, and make predictions and targets the same shape before you compare them.

- **PyTorch and NumPy default floats**: PyTorch float32, NumPy float64. Convert with torch.as_tensor(X, dtype=torch.float32) at the edge.
- **What shares memory**: torch.from_numpy, view and slices share memory. torch.tensor copies. In-place ops on a shared tensor change the source.
- **(n, 1) against (n,)**: Broadcasts to (n, n) without an error. Flatten with squeeze(1) or reshape(-1) and assert shapes.
- **Shape after sum(dim=1, keepdim=True) on (4, 3)**: (4, 1). Without keepdim: (4,).

B23-L2 Autograd, gradients from the chain rule

backward() fills .grad on leaf tensors. Update in place under no_grad, reset .grad, and check any hand-written gradient with central differences.

- **What backward() does**: Applies the chain rule from a scalar loss and stores gradients in .grad of leaf tensors (created with requires_grad=True).
- **Safe manual update**: with torch.no_grad(): w -= lr * w.grad, then w.grad.zero_().
- **Gradient of mean squared error for a linear model**: 2 / n * X.T @ (X @ w - y).
- **Central-difference check**: (f(w + h) - f(w - h)) / (2 * h) per weight. Disagreement beyond about 1e-6 means the formula is wrong.

B23-L3 Gradient descent, learning rates and optimisers

Step against the gradient, scale features so one rate fits all weights, watch the loss curve, and let an optimiser such as AdamW hold the update rule.

- **The gradient descent update**: w = w - lr * grad. Against the gradient, scaled by the learning rate.
- **Signs of a bad learning rate**: Too high: loss grows or turns into nan. Too low: loss barely falls.
- **One optimiser step in PyTorch**: optimizer.zero_grad(), loss.backward(), optimizer.step().
- **AdamW and SGD defaults in PyTorch 2.15**: AdamW weight_decay 0.01, SGD weight_decay 0. A good start: AdamW with lr 1e-3.

B23-L4 Models as nn.Module, layers, activations and parameters

Subclass nn.Module, call `super().__init__()` first, register layers in attributes, Sequential or ModuleList, and put activations between Linear layers.

- **Skeleton of a PyTorch model**: Subclass nn.Module, call `super().__init__()` first, build layers in `__init__`, use them in forward, call model(x).
- **nn.Linear(in, out) weight shape and parameter count**: Weight (out, in), bias (out,). Parameters: in * out + out.
- **Why put ReLU between Linear layers**: Two linear layers equal one. A non-linear activation lets depth fit curves and interactions.
- **Layers in a plain Python list**: Not registered: parameters() misses them and they never train. Use nn.ModuleList or nn.Sequential.

B23-L5 Loss functions, logits and target shapes

Regression: MSELoss on matching shapes. Binary: logits into BCEWithLogitsLoss with float targets and no sigmoid in the model. Squeeze (n, 1) outputs first.

- **Loss for a numeric target**: nn.MSELoss() on predictions and targets of the same shape. Squeeze (n, 1) outputs first.
- **Loss for a 0 or 1 target**: nn.BCEWithLogitsLoss() on raw logits and float targets. No nn.Sigmoid() in the model.
- **pos_weight for an imbalanced binary target**: torch.tensor([n_neg / n_pos]), computed on training rows.
- **Stable binary loss formula**: mean(max(z, 0) - z * y + log1p(exp(-|z|))).

B23-L6 Dataset, DataLoader and preprocessing fitted on training rows

A Dataset returns rows, a DataLoader batches them, training shuffles and validation does not, and every preprocessing statistic comes from training rows only.

- **The two methods of a Dataset**: `__len__` returns the row count. `__getitem__(i)` returns row i as tensors, converted once in `__init__`.
- **DataLoader batches for 10 rows, batch_size=4**: 4, 4 and 2. drop_last=True gives 4 and 4.
- **Which loaders shuffle**: Training: shuffle=True with a seeded generator. Validation and test: no shuffle.
- **Preprocessing rule**: Fit means, standard deviations and categories on training rows only. Guard std 0, impute NaN first.

B16 PyTorch hygiene

B16-L1 Choose the device once, move model and data to it

Pick the device once and pass it in. Reassign tensors after .to(). Never hardcode "cuda".

- **Choose a device**: Once, at the edge: cuda if available, then mps, then cpu. Pass it in.
- **Tensor.to vs Module.to**: Tensor.to returns a new tensor (reassign). Module.to moves in place.
- **GPU tensor to NumPy**: t.detach().cpu().numpy().
- **Hardcoded .cuda()**: Breaks on CPU machines and CI. Use .to(device).

B16-L2 A training loop that does what you think

zero_grad, train and eval modes, inference_mode for validation, logits into CrossEntropyLoss, loss.item() for logs, and early stopping on validation loss.

- **The training iteration**: to(device), zero_grad, forward and loss, backward, step.
- **Validation mode**: model.eval() and torch.inference_mode(). Back to model.train() after.
- **CrossEntropyLoss input**: Raw logits. It applies log-softmax itself.
- **Early stopping**: Stop when validation loss has not improved by min_delta for patience epochs. Keep the best weights.

B16-L3 Reproducible PyTorch runs and safe checkpoints

Seed every generator at the entry point, give DataLoaders their own generator, and checkpoint state dicts, not objects.

- **Seed a PyTorch run**: random.seed, np.random.seed, torch.manual_seed at the entry point, plus a seeded Generator for each DataLoader.
- **torch.load default in PyTorch 2.14**: Weights only. Pickled objects fail with UnpicklingError.
- **Checkpoint contents**: Model and optimizer state dicts, epoch, RNG state, best metric.
- **Deterministic operations**: torch.use_deterministic_algorithms(True) raises on operations with no deterministic version.